



Systemes paralleles et distribues

## Projet : Parallel Ant Foraging

Javier andres TARAZONA JIMENEZ

Jose Daniel CHACON

**Encadrant :** JUVIGNY Xavier

## Table des matières

---

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>2</b> |
| <b>2</b> | <b>Analyse du problème</b>                                       | <b>2</b> |
| 2.1      | Objectifs de la simulation . . . . .                             | 2        |
| 2.2      | État de l'art . . . . .                                          | 2        |
| 2.3      | Profilage et identification des goulots d'étranglement . . . . . | 3        |

## 1 Introduction

---

## 2 Analyse du problème

---

### 2.1 Objectifs de la simulation

L'objectif principal de la simulation est de modéliser le problème de fourragement d'une colonie de fourmis artificielles sur un terrain 2D hétérogène, afin d'identifier des trajets efficaces entre la fourmilière et une source de nourriture. Le cadre retenu suit un modèle d'intelligence en essaim où des agents simples produisent un comportement collectif émergent à partir de règles locales.

Le modèle vise à reproduire une coordination indirecte entre agents via deux types de phéromones : un phéromone d'exploration vers la nourriture ( $V_1$ ) et un phéromone de retour vers le nid ( $V_2$ ). Chaque fourmi évolue avec deux états (*unloaded*/*loaded*) et applique une politique exploration-exploitation pilotée par le paramètre  $\varepsilon$ , avec des transitions d'état lors de l'arrivée sur la nourriture ou sur la fourmilière.

L'environnement a pour objectif de représenter une difficulté de déplacement réaliste : chaque cellule de la carte possède un coût de traversée dans  $[0, 1]$ , et les zones non franchissables sont traitées comme obstacles. Le paysage est généré de manière fractale afin d'introduire des gradients de difficulté non uniformes et de contraindre la dynamique des déplacements.

Enfin, la simulation poursuit un objectif d'évaluation fonctionnelle au niveau de la colonie : la quantité de nourriture rapportée au nid sert d'indicateur global de performance. Ce cadre constitue la base expérimentale du projet, avant les étapes d'optimisation algorithmique (vectorisation et parallélisation) décrites dans le contexte général.

### 2.2 État de l'art

L'implémentation actuelle initialise d'abord l'environnement dans `fractal_land.cpp` en construisant une carte fractale par raffinement récursif de sous-grilles. Dans `ant_simu.cpp`, cette carte est ensuite normalisée pour ramener les altitudes dans l'intervalle  $[0, 1]$ , ce qui fixe le coût de déplacement utilisé par les fourmis pendant la simulation.

La population est ensuite instanciée avec un nombre fixe de fourmis, des positions initiales pseudo-aléatoires et des paramètres globaux du modèle (*exploration*, *bruit*, *évaporation*). La structure `pheromone` (dans `pheromone.hpp`) maintient deux valeurs de phéromones par cellule, un tampon de mise à jour, ainsi que des bordures fantômes marquées à  $-1$  pour empêcher les traversées hors domaine.

À chaque itération, la boucle principale de `ant_simu.cpp` traite les événements SDL puis exécute `advance_time`. Cette routine appelle `ant::advance` pour chaque agent, applique l'évaporation globale des phéromones (`do_evaporation`) puis valide l'état suivant de la carte (`update`). L'évolution individuelle d'une fourmi, définie dans `ant.cpp`, combine un choix local de voisin (aléatoire ou guidé par le maximum de phéromone), l'accumulation d'un budget de mouvement (`consumed_time < 1`), le marquage local des phéromones et la mise à jour de l'état chargé/non chargé.

Le rendu de l'état simulé est assuré par `renderer.cpp` et `window.cpp`. Le rendu affiche le terrain, la position des fourmis, la distribution des deux phéromones et la courbe de nourriture cumulée. La couche fenêtre/rendering SDL est construite avec mécanismes de repli (*fallback*) pour conserver l'exécution même lorsque certaines options matérielles ne sont pas disponibles.

## 2.3 Profilage et identification des goulots d'étranglement

Le profilage a pour objectif d'isoler les coûts dominants de la simulation avant toute optimisation, en particulier avant les étapes de vectorisation et de parallélisation. Nous présentons ici la définition des blocs à mesurer, instrumentation temporelle, protocole d'exécution reproductible et structure des résultats attendus. Cette section décrit donc la méthode de profilage et les objets techniques mobilisés, sans présenter de résultats chiffrés.

TABLE 1 – Découpage des parties mesurées pour le profilage de la simulation

| ID | Nature           | Rôle                                             | Périmètre            |
|----|------------------|--------------------------------------------------|----------------------|
| P0 | Initialisation   | Génération fractale du terrain                   | One-shot (démarrage) |
| P1 | Initialisation   | Normalisation des coûts du terrain               | One-shot (démarrage) |
| P2 | Initialisation   | Création et placement de la population           | One-shot (démarrage) |
| K0 | Noyau simulation | Coût total du noyau par itération                | Itératif             |
| K1 | Noyau simulation | Avance globale de toutes les fourmis             | Itératif             |
| K2 | Noyau simulation | Logique interne hors marquage phéromones         | Itératif             |
| K3 | Noyau simulation | Marquage local des phéromones                    | Itératif             |
| K4 | Noyau simulation | Évaporation globale des phéromones               | Itératif             |
| K5 | Noyau simulation | Validation/swap de la carte de phéromones        | Itératif             |
| R0 | Rendu SDL        | Affichage (terrain, fourmis, phéromones, courbe) | Itératif, hors noyau |
| E0 | Événements SDL   | Traitement de <code>SDL_PollEvent</code>         | Itératif, hors noyau |

Les relations structurelles utilisées pour l'analyse des goulots sont explicites :

$$K0 = K1 + K4 + K5$$

$$K1 = K2 + K3 + \text{reste}(K1)$$

Les blocs *P0*, *P1* et *P2* sont des coûts *one-shot* d'initialisation et ne doivent pas être mélangés aux métriques par itération. Les blocs *R0* (rendu SDL) et *E0* (gestion des événements SDL) sont mesurés séparément et traités comme non parallélisables dans l'analyse du noyau de calcul.

La stratégie instrumentale côté C++ s'appuie sur `timing_profile.hpp` et `timing_profile.cpp`. Le fichier `timing_profile.cpp` instancie l'état global de profilage (`g_timing_profile_totals`, `g_timing_profile_enabled`), tandis que `timing_profile.hpp` fournit l'horloge monotone (`profile_now_ns`) et les accumulateurs fins (`profile_add_k2`, `profile_add_k3`). L'accumulation des durées est effectuée bloc par bloc dans la boucle benchmark du simulateur, ce qui permet de séparer coûts d'initialisation, coûts noyau et coûts hors noyau.

L'automatisation est assurée par `scripts/measure_temps.sh`. Le pipeline suit une séquence reproductible : compilation release, puis 5 répétitions d'exécution avec 1200 itérations par répétition, 200 itérations de *warm-up* exclues et 1000 itérations effectivement mesurées, en configuration `OMP_NUM_THREADS=1`. Le script consolide les artefacts méthodologiques (`raw_measurements.csv`, `init_times.csv`, `table_a.csv`, `table_b.csv`, `amdahl_prediction.csv`, `summary.md`) afin de préparer la lecture des goulots.

La méthode de lecture priorise les métriques `ms/iter` et % noyau pour identifier les *hotspots* dominants. Les conclusions doivent d'abord cibler les blocs du noyau (*K0–K5*), puis isoler l'impact des composants hors noyau (*R0*, *E0*) pour éviter les biais d'interprétation.